Networkeando RD

Informe de la migración a Next.js + SQLite · 2 de noviembre de 2026

Resumen en pocas palabras

La app pasó de ser un solo archivo `index.html`, que solo funcionaba dentro de claude.ai, a un proyecto **Next.js con TypeScript**. Ahora tiene su propia **base de datos SQLite** y corre en tu laptop sin depender de ningún servicio externo. El diseño, los textos y las animaciones se mantuvieron iguales.

Estado: los pasos 1 a 5 están hechos, probados y guardados en git (un commit por paso). La app está corriendo en `http://localhost:3000`.
Pendiente: el paso 6 (revisión de diseño con las skills), poner tu clave de Anthropic si quieres la IA, y una prueba con celulares reales.

Dónde está el proyecto: `C:\Users\Analista BPA\networkeando-rd`. Lo saqué del zip a su propia carpeta porque Claude Code estaba abierto en otro proyecto (`simulador-precio`), y así no se mezclan.

1. Lo que se instaló en tu computadora

Programas

Qué	Versión	Para qué sirve	Cómo se instaló
Node.js (incluye npm)	v26.13.0 LTS · npm 11.20.0	Ejecuta JavaScript fuera del navegador. Next.js lo necesita para funcionar.	Instalador oficial de nodejs.org en `C:\Program Files\nodejs\`. Antes de instalarlo comprobé su huella digital (SHA-256) contra la que publica Node. winget no sirvió porque la red bloquea su catálogo.

Skills de Claude Code

Copié las 8 skills que venían en el zip (`.claude/skills`) a tu carpeta personal `C:\Users\Analista BPA\.claude\skills`, para que estén disponibles en cualquier proyecto. También siguen dentro del proyecto.

Skill	Para qué sirve
impeccable	Auditoría y pulido de diseño (`/impeccable audit`, `polish`, `critique`).
review-animations	Revisa animaciones con el criterio de Emil Kowalski.
improve-animations	Auditoría de animaciones y planes para mejorarlas.
animate	Crear animaciones nuevas paso a paso.
emil-design-eng	Criterio de diseño e interacción de Emil Kowalski.
design-taste-frontend (taste-skill)	Criterio visual para evitar diseños genéricos.
ui-ux-pro-max	Catálogo de estilos, paletas y tipografías.
huashu-design	Prototipos HTML, presentaciones y animaciones exportables.

Librerías del proyecto (dentro de `node_modules`, solo para esta app)

Paquete	Versión	Para qué sirve
next	16.4.4	El framework: páginas, servidor y rutas de API.
react, react-dom	19.3.1	Para construir la interfaz con componentes.
qrious	4.0.2	Dibuja el código QR. Es la misma librería que usaba el original.
@prisma/client · prisma	6.19.3	Crea la base de datos SQLite y permite leerla y escribirla desde TypeScript. Usé la 6 porque la 7 necesita compilar un módulo nativo en Windows.
@anthropic-ai/sdk	1.3.0	Librería oficial para llamar a Claude desde el servidor (match con IA).
typescript	5.9.3	Revisa los tipos del código. Usé la 5 porque Next todavía no es compatible con la 7.
@types/react, @types/react-dom, @types/node	19 / 19 / 22	Descripciones de tipos para que TypeScript entienda React y Node.

npm 11 bloquea los scripts de instalación hasta que alguien los aprueba. Solo aprobé los de los tres paquetes oficiales de Prisma, que descargan el motor de la base de datos. Quedó anotado en `package.json` → `allowScripts`.

2. Lo que hice en cada paso

Paso 1 · Probar el original `hecho · commit 2c7e0ec`

- Serví `index.html` con `python -m http.server` y probé cada vista en escritorio y en celular.
- El modo `#demo` funciona: búsqueda animada, modal "¡Hicieron match!" con Ana Lucía (98%) y 6 matches más.
- Anoté cómo se ve y cómo se comporta cada vista en `REFERENCIA.md`, para comparar después.

Paso 2 · Next.js con React y TypeScript `hecho · commit 1588feb`

- Moví el original a `legacy/index.html` como referencia.
- Saqué las imágenes de los personajes (DUO_L y DUO_R) con un script a `public/duo-l.webp` y `public/duo-r.webp`.
- Copié el CSS original tal cual a `app/globals.css`: colores de `:root`, animaciones, `prefers-reduced-motion` y safe-area. La fuente Manrope ahora se carga con `next/font`.
- Dividí la interfaz en componentes: `Barra`, `VistaInicio`, `VistaPerfil`, `VistaClics`, `VistaProyectar`, `MatchModal`, `Buscando` y piezas pequeñas en `ui.tsx`. Se mantienen todos los `aria-*`, el foco del modal y la tecla Escape.
- La lógica del match (`limpiar`, `inferir`, `CLAVES`, `analizar`…) está en `lib/match.ts`, igual que en el original.
- Las vistas siguen cambiando por hash: `#inicio`, `#perfil`, `#clics`, `#proyectar`, `#demo`.
- **Probado:** la demo se ve y se comporta igual que el original (mismos textos, mismos porcentajes, clave CARIBE-26), en escritorio y en celular.

Paso 3 · Base de datos SQLite con Prisma `hecho · commit 4cfe89b`

- Tabla `Perfil` con los mismos campos que `limpiar()`: idea, needs, power, anon, alias, que, contact, likes, color, createdAt. El archivo de la base es `prisma/dev.db`.
- **Rutas de API:**
  - `GET /api/perfiles`: los 800 más recientes primero.
  - `PUT /api/perfiles/[id]`: crea o actualiza tu perfil.
  - `DELETE /api/perfiles/[id]`: lo borra.
- **Identidad:** la primera vez se crea un id anónimo al azar y se guarda en una cookie (`nw_uid`) que JavaScript no puede leer. Solo puedes editar y borrar el perfil de tu cookie; si intentas tocar otro, la respuesta es 403.
- **Validación en el servidor** (`lib/validar.ts`): revisa tipos y largos máximos (idea 280, superpoder 140, nombre 40, a qué te dedicas 50, contacto 80) y rechaza lo inválido con un error 400. La fecha de creación la pone el servidor.
- El `onSnapshot` se reemplazó por una consulta cada 5 segundos. Si nada cambió, la pantalla no se redibuja.
- El `localStorage` de "clic-notificados" quedó igual.
- **Probado con dos personas** (el navegador y otra persona simulada con curl, cada una con su cookie): se ven entre sí y el match funcionó (96%). Un tercer perfil apareció solo en menos de 5 segundos. También probé borrar y los permisos 403 y 400.

Paso 4 · IA para el match hecho · commit 2a14021

- `POST /api/match` llama a Claude (modelo `claude-opus-5-5`, esfuerzo bajo) y pide la respuesta en JSON con un esquema fijo. Las instrucciones son las mismas del original.
- La clave va en `.env.local` (`ANTHROPIC_API_KEY=`), que git ignora. Nunca llega al navegador: lo comprobé buscándola en el código que se envía al navegador.
- Activé el respaldo automático de Anthropic (`fallbacks: "default"`): si el modelo se niega a responder por sus filtros de seguridad, otro modelo responde en la misma llamada.
- Límite de 12 pedidos por minuto por persona, para proteger tu clave.
- **Probado:** sin clave, la app usa el método básico (`inferir` + `CLAVES`). Con una clave inválida, la IA falla, la app avisa "La IA no respondió a tiempo…" y usa el método básico.

Paso 5 · QR y uso en el evento hecho · commit 7d40201

- El QR ya no apunta a claude.ai: usa la dirección donde corre la app. Si la abres como `localhost`, la app averigua la IP de tu laptop en el Wi-Fi (hoy `10.13.17.42`) para que el QR les sirva a los celulares.
- Si un día la publicas en internet, puedes fijar la dirección con la variable `NEXT_PUBLIC_SHARE_URL`.
- Nuevo script `npm run evento`: prepara la base, construye la versión rápida (producción) y la deja abierta a la red.
- **Probado:** la versión de producción se construye sin errores y funciona entrando por la IP, como lo haría un celular.

Paso 6 · Revisión de diseño pendiente

Todavía no lo hice. Las skills ya están instaladas. Cuando me digas, corro `/impeccable audit` y `review-animations` y te doy una lista ordenada de mejoras, sin aplicar ninguna hasta que la apruebes.

3. Cómo correr la app

Para trabajar en ella (modo desarrollo)

```
cd "C:\Users\Analista BPA\networkeando-rd"
npm run dev
```

Abre `http://localhost:3000`. Para ver la demo: `http://localhost:3000/#demo`. Para pararla: `Ctrl + C` en la terminal.

Para el evento (los celulares se conectan por el mismo Wi-Fi)

1. Si tienes `npm run dev` abierto, ciérralo primero, porque los dos usan el puerto 3000.
2. Corre:

   ```
   cd "C:\Users\Analista BPA\networkeando-rd"
   npm run evento
   ```

3. La primera vez, Windows pregunta si Node.js puede usar la red: marca **Redes privadas** y acepta. Si no aceptas, los celulares no podrán entrar.
4. En la laptop abre `http://localhost:3000/#proyectar` y toca "Pantalla completa". El QR ya apunta a tu laptop.
5. Prueba primero con tu celular: escanea el QR y crea un perfil.
6. Durante el evento, no cierres la terminal ni dejes que la laptop se duerma (conéctala a la corriente).

> **Ojo con el Wi-Fi:** muchas redes de invitados o corporativas tienen "aislamiento de clientes" y no dejan que un celular vea a la laptop. Si pasa, usa un router normal o el hotspot de un celular. La IP de la laptop cambia según la red; el QR la actualiza solo cuando abres `#proyectar`.

Para activar la IA

Abre `.env.local`, pega tu clave después de `ANTHROPIC_API_KEY=` y vuelve a arrancar la app. Sin clave, todo funciona con el método básico.

Para empezar un evento desde cero

Para la app, borra `prisma\dev.db` y corre `npx prisma migrate dev`. Ahora mismo la base está vacía: borré los perfiles que creé para las pruebas.

4. Hosting gratuito con SQLite (sin publicar nada todavía)

SQLite guarda todo en un archivo, así que necesitas un hosting con **disco permanente**. Si el disco se borra en cada reinicio, se pierden los perfiles. Estos planes cambian seguido: confirma precios y límites en cada página antes de decidir.

Opción	¿Gratis?	¿Sirve con SQLite?	Comentario
Tu laptop en el Wi-Fi del evento	Sí	Sí, tal como está	Lo que ya tienes. No necesita cuenta ni cambios. Solo funciona dentro del lugar.

Turso + Vercel	Tienen plan gratis	Sí: Turso es SQLite en la nube (libSQL)	Vercel no guarda archivos, así que la base iría en Turso. Hay que agregar el adaptador de Prisma para libSQL y cambiar la conexión. Es un cambio pequeño.
Railway	Crédito de prueba; después, plan pagado bajo	Sí, con un "volume" (disco permanente)	Fácil de usar, pero no es gratis de forma permanente.
Render	El plan gratis no tiene disco permanente	No en el plan gratis	El servicio gratis se duerme y su disco se borra; el disco permanente es de pago.
Fly.io	Ya no tiene plan gratis para cuentas nuevas	Sí, con volumen	Bueno técnicamente, pero de pago.

Mi recomendación: para un evento presencial, usa **tu laptop** (`npm run evento`): es gratis, ya funciona y los datos se quedan contigo. Si después quieres que la app viva en internet, la opción gratuita más sólida es **Vercel + Turso**, pero necesita ese cambio pequeño en la conexión a la base. Te lo puedo preparar cuando quieras, sin publicar nada sin tu permiso.

5. Cosas que conviene saber

- **Sin probar con celulares reales.** Probé entrando por la IP desde la misma laptop. La prueba final es escanear el QR con tu celular en el Wi-Fi del evento. Tampoco pude leer el QR con un programa (no se permitió cargar código externo en la página), así que confirma con la cámara que abre la dirección correcta.
- **IA en la demo:** al entrar a `#demo`, la primera búsqueda puede empezar antes de saber si hay IA y usar el método básico; "Buscar de nuevo" ya usa la IA. El original tenía el mismo comportamiento.
- **Aviso de npm audit:** 3 avisos "high" en `deepmerge-ts`, que usa la herramienta de línea de comandos de Prisma 6 (no la app que ven los usuarios). Se arregla subiendo a Prisma 7, que es un cambio grande. Lo dejé así a propósito.
- **CLAUDE.md** todavía describe el plan viejo (Firebase/Supabase). Conviene actualizarlo para que futuras sesiones de Claude Code no se confundan.
- El círculo con una "N" abajo a la izquierda solo aparece en modo desarrollo (es la herramienta de Next.js). En `npm run evento` no sale.
- Next.js creó solo un archivo `AGENTS.md` con notas para asistentes de IA. Es normal.

6. Historial de git

Commit	Mensaje
`2c7e0ec`	Paso 1: proyecto original y notas de referencia
`1588feb`	Paso 2: pasar la app a Next.js con React y TypeScript
`4cfe89b`	Paso 3: base de datos SQLite con Prisma y rutas de API
`2a14021`	Paso 4: elegir el match con IA desde el servidor
`7d40201`	Paso 5: QR con la dirección real y uso en el evento

Git no guarda `.env`, `.env.local` (tu clave), `prisma/dev.db` (los perfiles) ni `node_modules`.